

Informática II

Tipos estructurados de datos: Punteros

Profesor Dr. Paul Bustamante

Índice

- 1. Introducción a los punteros**
- 2. Punteros y Arrays**
- 3. Indexación de un puntero**
- 4. Asignación dinámica de memoria**
- 5. Array de punteros**

1. Punteros - Introducción

El valor de cada variable está almacenado en una determinada posición de la memoria del ordenador, caracterizado por una ***dirección*** (que se suele expresar en un valor hexadecimal). Por lo general, gracias a los nombres de las variables, no es necesario que el programador sepa cuál es la dirección de sus variables. Sin embargo, en ciertas ocasiones es más útil trabajar con las direcciones que con los propios nombres.

El C++ dispone de un ***operador de dirección "&"*** que nos permite determinar la dirección de una variable, y de un tipo especial de ***variables destinadas a contener direcciones de variables***. Estas variables se llaman ***punteros o apuntadores (pointers)***.

Además, C++ dispone de un ***operador de indirección "*"*** que permite acceder al valor depositado en la zona de memoria, a la que apunta el puntero.

Así pues, un ***puntero*** es una variable que puede contener la dirección de otra variable.

Un ***puntero*** ocupa por lo general 4 bytes de memoria y se debe declarar o definir de acuerdo con el tipo del dato al que apunta.

Así, un puntero del tipo ***int*** se declara de la siguiente manera: ***int *p;*** ó ***int* p;***

Mientras que uno del tipo ***float***: ***float *x1;***

No se deben mezclar los tipos, debido a que ocupan diferentes bytes dentro de la memoria.

Los punteros también tienen su propia dirección de memoria.

1. Punteros - Introducción

```
int q, *p;  
q = 144;                               // asigna 144 a q  
p = &q;                                // asigna a p la dirección de q  
cout << q << " " << hex << &q << endl; // q y su dirección  
*p = 441;                               // cambiamos lo que hay en q, a  
cout << q << endl;                   // través de p
```

Algunas notas sobre punteros:

- Las constantes o expresiones no tienen dirección.
- Las direcciones de una variable no se pueden cambiar.
- Aunque se puede convertir un puntero de un tipo a otro usando un **cast**, no se debe hacer, ya que daría error en tiempo de ejecución, sino coinciden el # de bytes.

Ejemplo:

```
p = &q;           //correcto  
p = &34;          //las constantes no tienen dirección  
p = &(q*2);       //las expresiones no tienen dirección  
&q = p;           //las direcciones no se pueden cambiar
```

1. Punteros – Ejemplo

```
// my first pointer
#include <iostream>
using namespace std;

void main (){
    int firstvalue, secondvalue, thirdvalue;
    int * mypointer;
    thirdvalue = 5;

    mypointer = &firstvalue;
    *mypointer = 10;
    mypointer = &secondvalue;
    *mypointer = 20;
    mypointer = &thirdvalue;
    *mypointer = 30;
    cout << "firstvalue is " << firstvalue << '\n';
    cout << "secondvalue is " << secondvalue << '\n';
    cout << "thirdvalue is " << thirdvalue << '\n';
}
```

Ejemplo3-1

1. Punteros – Ejemplo

```
// more pointers
#include <iostream>
using namespace std;

void main () {
    int firstvalue = 5, secondvalue = 15;
    int * p1, * p2;

    p1 = &firstvalue;
    p2 = &secondvalue;
    *p1 = 10;
    *p2 = *p1;
    p1 = p2;
    *p1 = 20;

    cout << "firstvalue is " << firstvalue << '\n';
    cout << "secondvalue is " << secondvalue << '\n';
}
```

Ejemplo 3-2

2. Punteros y Arrays

En C++ los punteros y arrays están estrechamente relacionados. De hecho, cualquier operación que se haga con *arrays* se puede hacer también con los punteros.

Ejemplo:

Supongamos que tenemos un array de 20 enteros:

```
int datos[20]
```

Para acceder a los elementos del array usando un *puntero*, éste debe ser creado del mismo *tipo* al que se desee acceder y copiar la dirección de array en él:

```
int *ptr = datos;
```

Luego, para acceder a un elemento en concreto, se usará la siguiente nomenclatura:

```
*(ptr + indice) // índice es el elemento a acceder.
```

La otra forma de hacerlo es con el nombre del array directamente:

```
*(datos + indice)
```

2. Punteros y Arrays - Ejemplo

```
#include <iostream>
using namespace std;
void main(void)
{
    int datos[]={10,20,30,40,50}; //array de 5 enteros
    int numDatos = sizeof(datos)/sizeof(int); //int datos [5];
    int a;
    cout << "Numero de datos: " << numDatos << endl;
    cout << "Mostrar con [] el array\n" ;
    //acceder a los elementos con el índice
    for (int i=0;i<numDatos;i++)
        cout << i << ": " << datos[i]<<endl;
    //acceder con punteros
    int *p = datos;
    // Forma de acceder 1
    cout << "Mostrar con punteros el array: Forma 1\n" ;
    for (int i=0;i<numDatos;i++){
        cout << i << ": " << *(p+i) << endl;
    }
    // Forma de acceder 2
    cout << "Mostrar con punteros el array: Forma 2\n" ;
    for (int i=0;i<numDatos;i++){
        cout << i << ": " << p [i] << endl;
    }
    // Forma de acceder 3
    cout << "Mostrar con punteros el array: Forma 3\n" ;
    for (int i=0;i<numDatos;i++) {
        cout << i << ": " << *p << endl;
    }
}
```

Ejemplo 3-3

```
#include <iostream>
using namespace std;
void main ()
{
    int numbers[5];
    int * p;
    p = numbers;
    *p = 10;
    p++;
    *p = 20;
    p = &numbers[2];
    *p = 30;
    p = numbers + 3;
    *p = 40;
    p = numbers;
    *(p+4) = 50;
    for (int n=0; n<5; n++)
        cout << numbers[n] << ", ";
    cout << "\n";
}
```

Ejemplo 3-4

3. Indexación de un Puntero

En el ejemplo anterior ha podido ver **cómo acceder a un array a través de un puntero**. En C++ también se puede hacer lo inverso, es decir, acceder a un puntero como si se tratase de un array, usando los “[]”. A esto se le llama ***indexación de punteros***.

```
#include <iostream>
#include <string.h>           //para strlen()
using namespace std;
void main() {
    //str es un puntero tipo char que apunta a la
    // dirección de memoria donde está almacenado el mensaje
    char *str="Me gustan los punteros";
    cout << "Mensaje: " << *str << endl;
    //sacar el mensaje invertido
    char *ptr = str;
    int len=strlen(str);
    cout << "\nMensaje Invertido con puntero:";
    for (int i=len-1;i>=0;i--) cout << ptr[i];
    cout << endl;
    cout << "\nMensaje Invertido con char inicial:";
    for (int i=len-1;i>=0;i--) cout << str[i];
    cout << endl;
}
```

Ejemplo 3-5

4. Asignación dinámica de memoria

Hay dos **formas** principales por medio de las cuales un programa puede **almacenar información en la memoria del ordenador**:

1. **Por medio del uso de variables**, en cuyo caso la cantidad de memoria necesaria queda fijada en tiempo de compilación **y no puede ser cambiada durante la ejecución del programa**. Esto es, si se ha reservado memoria para un array de 10 elementos de tipo **int**, este array no puede cambiar durante la ejecución.
2. **Por medio del sistema de *asignación dinámica de memoria* de C++**, de tal manera que se va asignando memoria en tiempo de ejecución según se vaya necesitando.

Variables: C++ tiene 2 operadores para la asignación dinámica de memoria: **new** y **delete**. Sus formas generales son:

```
var_puntero = new tipo_variable;  
delete var_puntero;
```

Usando **new** para un vector, el tamaño del vector se sitúa entre corchetes. Con **delete** el contenido al que apunta el puntero es borrado, pero no el puntero en sí.

```
float *var1 = new float;  
*var1 = 4.5;  
delete var1;
```

Arrays: Para asignar memoria dinámicamente a un vector de “**n**” elementos, se utiliza el siguiente formato:

```
var_puntero = new  
    tipo_variable[n];  
delete [] var_puntero;
```

```
int n=100;           //numero de datos  
int *datos = new int[n]; //asigna  
if (datos==NULL) { /* error */ }  
for (int i=0;i<n;i++) datos[i] = i*2;  
delete [] datos;      //libera
```

4. Asignación dinámica de memoria

```
#include <iostream>
using namespace std;

void main ()
{
    // EJEMPLO CON VARIABLES
    int *var1 = new int;
    *var1 = 4.5;
    cout << "Direccion puntero: " << var1 << ". Valor almacenado: " <<
*var1 << endl;
    int a;
    cout << "Introduzca el valor de la variable";
    cin >> a;
    *var1 = a;
    cout << "Direccion puntero: " << var1 << ". Valor almacenado: " <<
*var1 << endl;
    delete var1;
}
```

Variables

Ejemplo 3-6

4. Asignación dinámica de memoria

Arrays Dinámicos

```
#include <iostream>
using namespace std;

void main () {
    // EJEMPLO CON VECTORES
    int n;//numero de datos
    cout << "Introduzca el numero de datos: " ;
    cin >> n;
    int *datos = new int[n]; //asigna
    if (datos==NULL) { ;/* de error*/ }
    for (int i=0;i<n;i++){
        datos[i] = i*2;
    }
    for (int i=0;i<n;i++) cout << i << " :" << datos[i] << endl;

    delete [] datos;//libera
    cout << "\n";
}
```

Ejemplo 3-7

4. Asignación dinámica de memoria

Ejemplo: ordenar números dados por el teclado, mediante asignación dinámica de memoria.

```
#include <iostream>
using namespace std;
void main(void)
{
    int num;    //numero de datos
    cout << "dar numero de datos a ordenar:" ;
    cin >> num;
    int *datos = new int[num];    //asignar memoria
    if (datos==NULL) { cout << "Error, no hay memoria";    return;    }
    //pedir valores
    for (int i=0;i<num;i++) {
        cout << "\nDar elemento: " << (i+1) << " ?";
        cin >> datos[i];
    }
    //ordenar de mayor a menor
    for ( int i=0;i<num-1;i++) { //empieza por el 1º
        for ( int j=i;j<num;j++) { //empieza por el 2º
            if (datos[i] < datos[j]){
                int tmp = datos[i];
                datos[i] = datos[j];
                datos[j]=tmp;
            }
        }
    }
    //resultados
    for ( int i=0;i<num;i++) cout << datos[i] << endl;
    delete [] datos;    //liberar memoria
}
```

Ejemplo 3-8

4. Asignación dinámica de memoria

Para asignar memoria dinámicamente a una **estructura**, hay que definir una variable puntero a dicha estructura y luego utilizar el operador **new** para la asignación. Para *acceder* a los elementos de dicha estructura se utiliza el operador “->”

```
alumno *alumno1 = new alumno;  
alumno1-> carnet = 900345;  
strcpy(alumno1-> name, "Juan Pablo");  
cout << "Alumno: " << alumno1->name  
cout << "Carnet: " << alumno1->carnet
```

Para la asignación dinámica de memoria de arrays de estructuras, se utiliza la forma general de los arrays, es decir, hay que declarar un puntero a la estructura y luego asignar con el operador **new** el tipo y número de elementos. Para acceder a cada elemento del array se utiliza el índice y para acceder a cada miembro de la estructura se utiliza el operador “.”:

nombre[*indice*].miembro1;

```
struct alumno{  
    char name[40];  
    long carnet;  
};  
int num=100;  
alumno *clase = new alumno[num];  
clase[0].carnet=900234;  
clase[num-1].carnet=900435;  
delete [] clase;
```

4. Asignación dinámica de memoria

```
#include <iostream>
using namespace std;

struct alumno{
    char name[40];
    long carnet;
};

void main ()
{
    alumno *alumno1 = new alumno;
    alumno1->carnet = 900345;
    strcpy_s (alumno1->name,"Juan Pablo");
    cout << "Alumno: " << alumno1->name << endl;
    cout << "Carnet: " << alumno1->carnet << endl << endl;
    int num=3;
    alumno *clase = new alumno[num];
    clase[0].carnet = 900234;
    clase[1].carnet = 900235;
    clase[2].carnet = 900236;
    for (int i=0; i<num; i++)
        cout << "Numero carnet alumno " << i << ": " << clase[i].carnet << endl;
    delete [] clase;
}
```

Ejemplo 3-9

5. Array de punteros

Los punteros se pueden disponer en arrays como cualquier otro tipo de datos. Probablemente el uso más habitual es para crear tablas de cadenas o matrices numéricas bidimensionales.

```
char *dia[]={ "Lunes", "Martes", "Miercoles",
              "Jueves", "Viernes", "Sabado", "Domingo" };
int filas=sizeof(dia)/sizeof(char*);
for(int f=0;f<filas;f++,cout << endl){
    int col = strlen(dia[f]);
    for(int c=col-1;c>=0;c--){cout <<
dia[f][c];
    }
}
```

```
int col=10,fil=20;
float **mat; //puntero a punteros
           //matriz de fil x col
mat = new float*[fil];
for (int i=0; i<fil;i++)mat[i] = new float[col];
for (int f=0;f<fil;f++)
    for(int c=0;c<col;c++)
        mat[f][c]=c+f*col;
```

Para crear un array de dos dimensiones hay tres formas de hacerlo:

- Conociendo de antemano las dimensiones: `float mat[10][10];`
- Conociendo sólo una dimensión: `float *mat[10];`
- Sin conocer las dimensiones: `float **mat;`

La **forma más común de definir los arrays bidimensionales** es definiendo un **puntero a punteros**, de la siguiente forma:

- Luego hay que **reservar memoria para almacenar las “f” filas de punteros** que luego guardarán la información del array de columnas: `nombre = new tipo*[f];`
- Finalmente, se debe **reservar memoria para que cada fila contenga “c” columnas**: `nombre[f] = new tipo[num_col];`

5. Array de punteros

El acceso de los elementos de un array bidimensional se puede hacer por medio de índices, utilizando los “[]” que habitualmente usamos para los array’s: dato[i][j];

```
//generación de números aleatorios, "f" filas y "c" columnas
#include <iostream>
using namespace std;
void main(void)
{
    int num,cols;
    int filas=10;
    float **dat;
    cout << "Num columnas:?";
    cin >> cols;
    dat = new float*[filas];           //reserva para vector de punteros
    for (int i=0;i<filas;i++) dat[i] = new float[cols]; //cada fila
    for ( i=0;i<filas;i++)             //llenar la matriz de datos aleat.
        for (int j=0;j<cols;j++)
            dat[i][j] = (float)rand()/32767.0;

    for ( i=0;i<filas;i++,cout << endl) //consola
        for (int j=0;j<cols;j++)
            cout << dat[i][j] << " ";

    for ( i=0;i<filas;i++) //liberar cada fila
        delete [] dat[i];

    delete [] dat;           //liberar el vector de punteros
}
```

Ejemplo 3-10